

00 — SUT RECON (Phase 0)

- **Sinh viên:** Ninh Văn Khải — MSSV **23127060**
- **SUT:** EShop (`eshop-sut/`) — Express 5 + SQLite + React/Vite
- **Thời điểm recon:** 2026-08-22 (9:01 PM, Asia/Ho_Chi_Minh)
- **Feature phụ trách:** FR-03 (Quên/Đặt lại mật khẩu) · FR-08 (Thanh toán) · FR-15 (Quản lý sản phẩm)

Trước khi viết bất kỳ dòng test nào, em dành hẳn một phase để đọc source của SUT và xác minh lại mọi thứ bằng request thật. Em làm vậy vì nếu em đoán selector hoặc đoán hành vi của API thì sau này test có pass cũng không chứng minh được điều gì.

Em coi tài liệu này là **input bắt buộc** cho mọi phase sau. Mọi selector và mọi giá trị kỳ vọng trong

test của em đều phải dẫn nguồn về một dòng trong các bảng dưới đây. Em tự đặt ra nguyên tắc là không đoán

selector, chỗ nào chưa chắc thì phải mở file JSX ra đọc.

1. Môi trường đã xác minh

Đầu tiên em kiểm tra xem môi trường đã sẵn sàng chưa. Em không tin vào tài liệu mà chạy lệnh để xác nhận từng thành phần một:

Thành phần	Lệnh xác minh	Kết quả thật
Backend Express	<code>curl http://localhost:3000/api/products</code>	200 + JSON 5 sản phẩm seed
Frontend-web (Vite)	<code>curl -o /dev/null -w "%{http_code}" http://localhost:5173/</code>	200
Frontend-admin (Vite)	<code>Get-NetTCPConnection -State Listen</code>	Ban đầu chưa chạy → đã <code>npm install</code> cho <code>frontend-admin</code> (thiếu <code>node_modules</code>), cần chạy <code>npm run dev</code> để Vite cấp port (dự kiến 5174)
Node	<code>node -v</code>	v22.20.0
Playwright	<code>npx playwright --version</code>	1.62.1

Em xin ghi chú thêm về port. File `run_servers.sh` không chỉ định port cho Vite, nên Vite sẽ tự cấp port tăng dần. Hiện tại web đang giữ cổng **5173**, còn admin sẽ nhận **5174** nếu 5173 đã bận. Để tránh phải sửa code mỗi lần port đổi, em cho test đọc giá trị thật từ biến môi trường trong

`automation/tests/utils/env.js` gồm `WEB_BASE_URL`, `ADMIN_BASE_URL` và `API_BASE_URL`.

Riêng backend thì bắt buộc phải chạy ở cổng 3000 và em không đổi được. Lý do là cả `frontend-web` lẫn `frontend-admin` đều hardcode thẳng địa chỉ `http://localhost:3000/api` vào source, em kiểm tra thấy ở `ForgotPassword.jsx:17`, `Checkout.jsx:30` và `App.jsx:4`.

2. Seed data (bị DROP và seed lại mỗi lần khởi động backend — `database.js:14-20`)

Tiếp theo em đọc file `database.js` để nắm xem hệ thống seed sẵn những dữ liệu gì, vì em cần biết mình có thể dựa vào dữ liệu nào và không nên dựa vào dữ liệu nào:

Bảng	Nội dung seed
users	admin@eshop.com / Admin123! (role admin), test@eshop.com / Test1234! (role user)
categories	1 Điện thoại · 2 Laptop · 3 Phụ kiện
products	id 1..5 — iPhone 15 Pro Max 30tr · Galaxy S24 Ultra 28tr · MacBook Pro M3 45tr · AirPods Pro 2 6tr · Keychron Q1 4tr
coupons	SAVE10 percent 10 min 300k · BIGBUY fixed 50k min 500k · VIP100 fixed 100k min 300k max 2 lượt · EXPIRED percent 20 hết hạn 2020-01-01

Từ phần seed data này, em rút ra bốn điều ảnh hưởng trực tiếp tới cách em thiết kế test:

- Mỗi lần restart backend là toàn bộ dữ liệu test bị xoá sạch, nên em **không** để test nào phụ thuộc vào dữ liệu do test khác tạo ra.
 - Mật khẩu được lưu ở dạng **plaintext**, em kiểm chứng ở `database.js:88-90` và `server.js:23`, tức là hệ thống hoàn toàn không hash mật khẩu.
 - Để các test không đụng nhau, em cho mỗi test tự tạo một tài khoản riêng với email ngẫu nhiên dạng `ts-<timestamp>-<rand>@eshop.test`.
 - Giỏ hàng phía backend, tức biến `userCarts` ở `server.js:14`, là một biến **in-memory**. Nó mất khi restart, không khử trùng lặp và cũng không trừ tồn kho, nên em không thể dựa vào nó để seed dữ liệu.

3. Bảng route ↔ selector ↔ API ↔ rủi ro

Đây là phần em đầu tư nhiều thời gian nhất. Với mỗi màn hình thuộc ba feature em phụ trách, em mở file JSX ra đọc để lấy đúng selector, ghi lại API mà màn hình đó gọi, và ghi luôn những rủi ro em nhận thấy. Nhờ bảng

này mà sau đó em viết test không phải đoán một selector nào.

3.1 FR-03 — Quên mật khẩu & Đặt lại mật khẩu

Feature này nằm ở route `/forgot-password`, source là `frontend-web/src/pages/ForgotPassword.jsx`. Em xin lưu ý một điểm về cấu trúc: màn hình này gồm 2 bước nhưng cả hai đều nằm trong **cùng một route**, hệ thống

chỉ đổi qua lại bằng state `step` chứ không điều hướng. Vì vậy em không thể dùng URL để biết mình đang ở bước nào.

Bước	Selector (nguồn)	API gọi	Rủi ro / bug candidate
B1 nhập email	<code>getByRole('textbox')</code> — input type=text, <label> KHÔNG có <code>htmlFor</code> (dòng 47-53) ⇒ <code>getByLabel()</code> không dùng được	POST <code>/api/forgot-password</code>	Trả <code>resetToken</code> thẳng trong response body
B1 submit	<code>getByRole('button', { name: 'Lấy mã OTP' })</code> (dòng 57)	—	Không rate-limit
B2 hộp OTP	<code>.bg-green-100</code> chứa Mã OTP của bạn là: <token> (dòng 68-70) → lấy token bằng regex <code>/(\d{4})/</code> trên text	—	Token lộ trên UI
B2 nhập OTP	textbox thứ nhất của step 2 (dòng 73-79)	—	Token chỉ 4 chữ số ⇒ 9000 tổ hợp
B2 mật khẩu mới	<code>page.locator('input[type="password"]')</code> (dòng 83-89)	—	—
B2 submit	<code>getByRole('button', { name: 'Đặt lại mật khẩu' })</code> (dòng 93)	POST <code>/api/reset-password</code>	Không kiểm tra policy mật khẩu ở backend
Kết quả	<code>alert()</code> cho cả thành công lẫn lỗi ⇒ bắt bằng <code>page.on('dialog')</code> (dòng 22, 33, 36)	—	Không có toast DOM để assert

● Đây là cái bẫy phía frontend mà em thấy quan trọng nhất của FR-03 (`ForgotPassword.jsx:26`):

```
const flawedStrongPasswordRegex = /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*\s)[A-Za-z\d\s]{8,}$/;
```

Em phân tích đoạn regex này như sau. Thông báo lỗi hiển thị cho người dùng nói rằng mật khẩu `_` "phải có KÝ

TỰ ĐẶC BIỆT" `_` nhưng regex thực tế lại đòi **khoảng trắng** qua `(?=.*\s)`, đồng thời **cấm** mọi ký tự đặc biệt vì character class chỉ cho phép `[A-Za-z\d\s]`.

Hệ quả là mật khẩu `NewPass123!` tuy mạnh và có ký tự đặc biệt thì lại **bị chặn**, trong khi

`New Pass 123` yếu hơn nhưng có khoảng trắng thì lại **được chấp nhận**. Em ghi nhận đây là **BUG-03-01**,

lỗi không khớp giữa thông báo và logic kiểm tra, và em sẽ cover nó bằng test case ở FR-03.

Em cũng xin lưu ý là cái bẫy này ảnh hưởng tới chính test của em: mọi mật khẩu hợp lệ mà em dùng trong luồng giao diện đều phải có khoảng trắng, nếu không thì test sẽ fail vì bị FE chặn chứ không phải vì SUT sai.

Sau đó em dùng curl để xác minh hành vi thật của backend:

```
POST /api/forgot-password {"email":"test@eshop.com"}
⇒ 200 {"message":"Mã đặt lại mật khẩu đã được tạo","resetToken":"5860"} ← token LỘ

POST /api/forgot-password {"email":"nobody-xyz@nope.com"}
⇒ 404 {"error":"User not found"} ← user enumeration

POST /api/reset-password {"email":"test@eshop.com","resetToken":"0000",...}
⇒ 400 {"error":"Invalid token or email"}
```

Từ những gì đọc và thử được, em liệt kê ra các bug candidate của FR-03 như sau:

(a) token đặt lại mật khẩu bị lộ thẳng trong HTTP response; (b) token chỉ có 4 chữ số nên hoàn toàn brute-force được; (c) token không có thời hạn, vì em kiểm tra `server.js:66-82` thì thấy code không hề lưu lại thời điểm tạo token; (d) hệ thống không giới hạn số lần gọi; (e) có thể dò được tài khoản nào tồn tại thông qua việc phân biệt mã 404; (f) backend không kiểm tra độ mạnh mật khẩu nên em đặt được mật khẩu chỉ là `1`; (g) mật khẩu lưu plaintext; (h) khi reset thì hệ thống **không** xóa `locked_until`, dẫn tới đổi mật khẩu xong rồi mà vẫn không đăng nhập được; và (i) regex phía FE sai chuẩn như em đã phân tích ở trên.

3.2 FR-08 — Thanh toán (Checkout)

Luồng thanh toán đi qua ba route theo thứ tự `/` → `/cart` → `/checkout`, tương ứng với các file

`Home.jsx`, `Cart.jsx`, `Checkout.jsx` và phần state dùng chung ở `context/CartContext.jsx`.

Màn	Selector (nguồn)	API gọi	Rủi ro / bug candidate
Home — thêm giỏ	<code>getByRole('button', { name: 'Thêm vào giỏ' })</code> trong card sản phẩm (<code>Home.jsx:110</code>)	— (client-side)	—
Cart trống	<code>getByText('Giỏ hàng của bạn đang trống')</code> (<code>Cart.jsx:24</code>)	—	—
Cart → checkout	<code>getByRole('button', { name: 'Tiến hành thanh toán' })</code> (<code>Cart.jsx:74</code>)	—	Chưa login ⇒ <code>alert()</code> + redirect <code>/login</code>
Checkout — tổng tiền	<code>getByRole('spinbutton')</code> — input <code>type=number</code> <code>editableTotal</code> SỬA ĐƯỢC (<code>Checkout.jsx:96-106</code>)	—	● Price tampering
Checkout — coupon	<code>getByPlaceholder('Nhập mã giảm giá...')</code> (<code>Checkout.jsx:115</code>)	POST <code>/api/apply-coupon</code>	● công thức percent sai
Checkout — áp dụng	<code>getByRole('button', { name: 'Áp dụng' })</code> (<code>Checkout.jsx:120</code>)	—	—
Checkout — xác nhận	<code>getByRole('button', { name: 'Xác Nhận Thanh Toán' })</code> (<code>Checkout.jsx:146</code>)	POST <code>/api/checkout</code>	Backend tin <code>total_amount</code> client
Thành công	<code>getByRole('heading', { name: 'Thanh toán thành công!' })</code> (<code>Checkout.jsx:70</code>)	—	—

● Với FR-08, em đã xác minh được ba lỗi lớn bằng curl. Em xin dán nguyên văn request và response:

```
# (1) Price tampering – backend tin tuyệt đối total_amount của client
POST /api/checkout Authorization: Bearer <user token>
{"total_amount":1,"shipping_address":null}
⇒ 200 {"message":"Checkout successful","orderId":1}

# (2) Coupon percent tính SAI DẤU – server.js:432 dùng (1 - discount_value) thay vì (discount_value/
POST /api/apply-coupon {"code":"SAVE10","total_amount":1000000}
⇒ 200 {"discount_amount":-9000000,"final_amount":1000000}
  ← giảm 10% nhưng tổng tiền TĂNG GẤP 10 LẦN

# (3) IDOR – GET /api/orders/:id KHÔNG có authenticateToken (server.js:340)
GET /api/orders/1 (không gửi header nào)
⇒ 200 {"id":1,"user_id":2,"total_amount":1,"status":"pending",...}
```

Ngoài ba lỗi trên, em còn đọc được từ source một số bug candidate khác của FR-08 mà em sẽ cần test để xác nhận:

- Hàm `clearCart` có được import ở `Checkout.jsx:8` nhưng em đọc kỹ thì thấy nó **không bao giờ được gọi**.

Như vậy giỏ hàng sẽ không bị xoá sau khi thanh toán, và người dùng có thể thanh toán lại lần nữa.

- Phía FE có gửi `items: cart` lên, nhưng backend ở `/api/checkout` (`server.js:302-317`) lại ****bỏ** qua hoàn

toàn trường `items` và chỉ lưu `total_amount` cùng `shipping_address`. Trong khi đó FE lại không gửi**

`shipping_address`, nên kết quả là mọi đơn hàng đều có địa chỉ giao hàng bằng `null`.

- Backend không kiểm tra trường hợp giỏ rỗng, cũng không kiểm tra khi `total_amount` là số âm hoặc không

phải là số.

- Hệ thống không trừ tồn kho, vì schema của bảng `products` vốn không có cột stock nào.
- Giỏ hàng được lưu trong **React Context, tức là in-memory**, chứ không phải localStorage

(`CartContext.jsx:7`). Reload trang là mất sạch giỏ. Điều này có nghĩa là với luồng giao diện, **em bắt

buộc phải thêm sản phẩm qua UI trong cùng một phiên** chứ không seed được qua API.

3.3 FR-15 — Quản lý sản phẩm (Admin)

Phần admin có cấu trúc khác hẳn hai feature trên: `frontend-admin` là một **SPA gói gọn trong một file**

duy nhất là `src/App.jsx` dài 922 dòng. Ứng dụng đổi màn hình bằng state `activeTab` và hoàn toàn không dùng

router. Điều này có nghĩa là em không thể điều hướng bằng URL, mà phải click đúng phần tử trên sidebar.

Bước	Selector (nguồn)	API gọi	Rủi ro
Login admin	<code>getByPlaceholder('Email') · getByPlaceholder('Password') · getByRole('button', { name: 'Login' })</code> (App.jsx:185-211)	POST /api/login	FE chặn <code>role !== 'admin'</code> bằng <code>alert('Bạn không phải là admin!')</code> (dòng 65)
Token	<code>localStorage['adminToken']</code> (dòng 68)	—	Có thể inject qua <code>addInitScript</code> để bỏ qua login UI
Vào tab SP	<code>getByText('Sản phẩm', { exact: true })</code> — là <code></code> không phải button (dòng 238-244)	—	—
Form thêm/sửa	<code>getByPlaceholder('Tên sản phẩm') (required) · getByPlaceholder('Giá tiền') (type=number) · getByPlaceholder('URL Ảnh') · getByPlaceholder('Mô tả') (textarea) · getByRole('combobox') (App.jsx:490-540)</code>	—	Giá tiền không required
Submit	<code>getByRole('button', { name: 'Lưu sản phẩm' })</code> (dòng 545)	POST hoặc PUT /api/products	—
Tiêu đề form	<code>getByText('Thêm sản phẩm mới') ↔ getByText('Sửa sản phẩm')</code> (dòng 486-488)	—	Phân biệt chế độ add/edit
Bảng SP	<code>getByRole('row').filter({ hasText: '<tên>' })</code> ; trong dòng có nút Sửa / Xóa (dòng 570-600)	DELETE /api/products/:id	—
Thông báo	Thêm mới: không alert , chỉ <code>refetch</code> . Sửa: <code>alert('Cập nhật thành công!')</code> . Lỗi: <code>alert('Lỗi lưu sản phẩm: ...')</code>	—	Bắt bằng <code>page.on('dialog')</code>

● Với FR-15, em cũng xác minh được một loạt lỗi lớn bằng request thật:

```
# (1) Thiếu access control HOÀN TOÀN — POST/PUT/DELETE /api/products không có authenticateToken
POST /api/products (KHÔNG gửi Authorization)
{"name":"RECON-NOAUTH","price":-5,...}
⇒ 200 {"message":"Product created","id":6} ← tạo được, giá ÂM cũng insert

DELETE /api/products/6 (KHÔNG gửi Authorization)
⇒ 200 {"message":"Product deleted"}

# (2) DELETE id không tồn tại vẫn báo thành công (server.js:186-191 không check this.changes)
DELETE /api/products/99999
⇒ 200 {"message":"Product deleted"}

# (3) Kiểu dữ liệu bất nhất — id CHẶN trả price dạng STRING (server.js:158)
GET /api/products/2 ⇒ {"id":2,...,"price":"28000000"} ← string
GET /api/products/1 ⇒ {"id":1,...,"price":30000000} ← number

# (4) id không tồn tại trả 200 {} thay vì 404 (server.js:157)
GET /api/products/9999 ⇒ 200 {}
```

● Nhưng bug phía frontend mà em thấy nặng nhất của FR-15 là lỗi "fake mass update" ở

App.jsx:107-112 :

```
await axios.put(`${API_URL}/products/${productForm.id}`, productForm);
const fakeMassUpdatedProducts = products.map((p) => ({
  ...p,
  name: productForm.name, // ← gán tên SP đang sửa cho TẤT CẢ sản phẩm
}));
setProducts(fakeMassUpdatedProducts);
```

Em đọc đoạn code này và thấy rằng sau khi bấm **Lưu sản phẩm** ở chế độ sửa, frontend gán tên của sản phẩm

đang sửa cho **toàn bộ** sản phẩm trong state. Kết quả là cả bảng đổi thành cùng một tên cho tới khi người dùng reload lại trang.

Điều làm em thấy lỗi này nguy hiểm là dữ liệu trong database thực ra vẫn đúng, chỉ có màn hình là sai. Admin

nhìn vào sẽ thấy toàn bộ sản phẩm giống hệt nhau và rất dễ bấm nhầm nút Xóa. Cách em bắt lỗi này là đếm số

dòng mang tên mới trong bảng, nếu lớn hơn 1 thì bug tồn tại.

Em còn ghi nhận thêm một số bug candidate khác của FR-15: em tạo được sản phẩm có `name` rỗng vì backend không validate; trường `price` dù là chuỗi, số âm hay số cực lớn thì vẫn được insert bình thường; `category_id` trỏ tới danh mục không tồn tại cũng vẫn insert được vì bảng không khai báo FOREIGN KEY nào.

Riêng về XSS thì em xin trình bày cẩn thận hơn một chút. Bảng ở trang admin render bằng cú pháp `{p.name}`,

mà React **có** tự escape nên payload không thực thi được ở màn hình này. Tuy nhiên payload độc vẫn được lưu nguyên vẹn vào database, và em tìm thấy những chỗ khác trong hệ thống có dùng `dangerouslySetInnerHTML`

— cụ thể là App.jsx:801 cho `shipping_address` và Home.jsx:64 cho ô tìm kiếm. Vì vậy em vẫn coi đây là rủi ro thật, chỉ là điểm phát nổ nằm ở màn hình khác.

4. Ngoài phạm vi (em đọc để biết mà tránh, không viết test)

Trong lúc đọc source, em có gặp một số chức năng nằm ngoài phạm vi ba feature được giao. Em liệt kê lại ở đây để tự nhắc mình không viết test lẫn sang phần của bạn khác:

Chức năng	Vì sao ngoài phạm vi
Import CSV <code>/api/admin/import-products</code>	Thuộc FR-16 — feature của thành viên khác
Quản lý danh mục / coupon / user (admin)	Thuộc FR-14 và các FR khác
Đăng ký / Đăng nhập (<code>/register</code> , <code>/login</code>)	FR-01 / FR-02 — chỉ dùng làm tiền đề cho FR-03 và FR-08, không viết test case tính điểm
<code>frontend-mobile</code> (FR-20)	Đề §5: Pool D không tính điểm HW04

Cuối cùng, em xin ghi lại ba cảnh báo mà em phát hiện trong lúc recon. Đây không phải là bug thuộc feature của em, nhưng nếu em không biết trước thì test của em sẽ fail mà em lại tưởng là do SUT sai.

Cảnh báo thứ nhất, về trang đăng nhập (`Login.jsx`). Trang login của web có khá nhiều lỗi hiển thị:

heading lại ghi `"Đăng Ký"`, label ghi `"Username"`, và ô mật khẩu để `type="text"`. Đáng chú ý nhất là nút submit có tên là **Sign In** chứ không phải **Đăng nhập** như em tưởng. Vì vậy Page Object đăng nhập của em phải dùng `getByRole('button', { name: 'Sign In' })`, và phải định vị hai ô nhập bằng **thứ tự textbox** chứ không dùng được `input[type=password]`, đơn giản vì phần tử đó không tồn tại ở trang này.

Cảnh báo thứ hai, về trang chi tiết sản phẩm (`ProductDetail.jsx:22-25`). Nút "Thêm vào giỏ hàng" ở trang này **cố tình bỏ qua cú click đầu tiên**, do đoạn code `clickCount === 0 → return`. Vì vậy khi làm luồng FR-08, em phải thêm sản phẩm từ trang **Home** thay vì đi qua `ProductDetail`.

Cảnh báo thứ ba, về API đăng nhập (`server.js:53`). Em phát hiện mỗi lần nhập sai mật khẩu thì

`login_attempts` tăng lên **2** chứ không phải 1. Như vậy chỉ cần sai **2 lần** là tài khoản đã bị khoá 3 phút. Điều này khiến em phải cẩn thận trong test FR-03 để không vô tình đăng nhập sai và tự khoá tài khoản của chính mình.

5. Kết luận Phase 0

Em xin tổng kết lại những gì đã làm được ở phase khảo sát này:

- ✅ Em đã xác nhận backend chạy ở cổng `3000` và frontend-web chạy ở cổng `5173`, kiểm chứng bằng `curl`

chứ không dựa vào tài liệu.

- ✅ Em đã xác minh **bằng request thật** được 9 hành vi lỗi của SUT, gồm: token bị lộ trong response, có thể

dò tài khoản tồn tại, sửa được giá khi thanh toán, công thức coupon tính nghịch dấu, xem được đơn hàng của

người khác qua IDOR, thao tác CRUD sản phẩm không cần đăng nhập, xoá một id không tồn tại vẫn báo thành

công, id chặn trả về giá dạng chuỗi, và id lạ trả về `200 {}` thay vì `404`.

-  Toàn bộ selector trong các bảng ở §3 em đều **đọc trực tiếp từ file JSX** và ghi kèm số dòng để dẫn nguồn.
-  Em phát hiện `frontend-admin` bị thiếu `node_modules` nên đã chạy `npm install` để cài. Em cũng đã chạy

`run_servers.sh` và xác nhận cổng thật của admin đúng là **5174** như dự kiến.

Sau khi hoàn thành phase này, em thấy đã đủ cơ sở để ****sang Phase 1 và bắt đầu thiết kế bảng test case với tối thiểu 12 case cho mỗi feature.****